Journal of Computational Science 82 (2024) 102424

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

Quasi-linear time heuristic to solve the Euclidean traveling salesman problem with low gap

Arno Formella

University of Vigo, Spain

ARTICLE INFO

MSC:
05C45
68T20
68W25
90C59

Keywords:
Euclidean TSP
Computational geometry
Heuristic
Approximation algorithm

ABSTRACT

The traveling salesman problem (TSP) is a well studied NP-hard optimization problem. We present a novel heuristic to find approximate solutions for the case of the TSP with Euclidean metric. Our pair-center algorithm runs in quasi-linear time and on linear space. In practical experiments on a variety of well known benchmarks the algorithm shows linearithmic (i.e., $\mathcal{O}(n \log n)$) runtime. The solutions found by the pair-center algorithm are very good on smaller problem instances, and better than those generated by any other heuristic with at most quadratic runtime. Eventually, the average gap of the pair-center algorithm on all benchmark instances with less than 1 001 points is 0.94% and for all instances with more than 1000 points up to 100 million points is 4.57%.

1. Introduction

The traveling salesman problem (TSP) has a long history [1,2]. It can be stated as a combinatorial optimization problem on a graph. Given an undirected graph $G = (V, E)$ with $n = |V|$ nodes and $m = |E|$ edges and an associated weight function $w : E \longrightarrow \mathbb{R}^{>0}$ that assigns to each edge a positive value, find a closed path (i.e., a tour) that visits all nodes of the weighted graph exactly once and has the smallest overall weight among all possible such tours. If we assign a value of 1 to all edges, the task reduces to finding a Hamilton cycle in the graph. To decide whether a Hamilton cycle exists in a graph is an NP-complete decision problem. Searching a tour with a weight of at most a certain value is NP-complete, too, because to check both conditions, i.e., it is a tour and its length is less than or equal to the given value, is easy. However, finding the shortest tour is only known to be NP-hard, as there is no known polynomial time algorithm that, given a tour, checks the condition that it is a shortest tour.

Additional requirements have been proposed to deal with the complexity of the general TSP with the ultimate goal to find efficient algorithms that solve the problem or provide at least good approximate solutions [2]. For instance, in the metric TSP (mTSP) it is required — regarding the edge weights — that the triangle inequality holds for all corresponding triples of nodes. Further, one might impose that the underlying graph is complete and the weight function reflects the Euclidean distances of the nodes embedded into a d-dimensional Euclidean space (eTSP), i.e., an eTSP is a special case of an mTSP. Deciding whether there exists a Hamilton cycle in a complete graph is trivial, however, searching the one with minimal tour length remains NP-hard even in the case of the Euclidean TSP.

More variations are available in the literature, for instance, the distance function can be defined on a torus or on a sphere. Other types include the asymmetric TSP where the underlying graph is directed [3] with newer ideas of how to solve in [4], the clustered TSP [5] where the point set is divided into clusters that must be visited in order, the maximum TSP [6] where the Hamilton cycle with maximal weight is searched for, the time windowed TSP [7] where the tour must meet additional arrival constraints at each node, precedence TSP [8–10] where there exist certain restrictions on the order of visits, or generalized TSP [11] where the point set is represented as a partition of disjoint subsets and the shortest tour with exactly one element of each subset is looked for, and many more [12]. Other modifications of TSP include the search for best open loops [13] or the search for a certain number of independent routes possibly starting at a certain fixed node always with the aim to minimize the overall length of the tours [14].

To solve the traveling salesman problem, a brute force exhaustive search would take $O(n!)$ time. The exact algorithm in [15,16] solves the general TSP with a runtime of $\mathcal{O}(n^2 2^n)$. For the Euclidean TSP, [17] presents an improvement to $2^{\mathcal{O}(n^{1-1/d})}$. The state-of-the art exact solver for the general TSP is still the branch-and-cut-and-price approach as implemented in Concorde [18]. A runtime estimation on random instances for Concorde is $\approx 0.21 \cdot 1.24^{\sqrt{n}}$ [19], but there are hard to solve instances for eTSP with a much higher runtime [20]. Recently,

E-mail address: formella@uvigo.gal.

https://doi.org/10.1016/j.jocs.2024.102424
Received 26 December 2023; Received in revised form 6 July 2024; Accepted 14 August 2024
Available online 17 August 2024

a quantum algorithm with runtime $\mathcal{O}(n^p 1.728^n)$ (for some constant p) for the general TSP was proposed in [21].

Due to its theoretical complexity and its practical importance many heuristics have been invented with the aim to provide at least a good approximation to the exact solution, i.e., one looks for algorithms that approximate the optimal solution by a factor $(1+\varepsilon)$ for a value of ε as small as possible. The value $(1+\varepsilon)$ is called the *approximation ratio* of the algorithm, ε when expressed in percent is called *excess*, or more frequently, *gap*. For the general TSP and even for the metric TSP, a $(1+\varepsilon)$-approximation remains NP-hard, whenever ε becomes smaller than a certain constant threshold $c > 0$ [22]. A solution to the mTSP can be approximated with a ratio of $3/2$ with the Christofides algorithm [23] in cubic time $\mathcal{O}(n^3)$. The work in [24] claims to improve this bound by a tiny factor. For the 2-dimensional Euclidean TSP however, [25] and, independently, [26] have found a polynomial time approximation scheme (PTAS) that runs in $\mathcal{O}(n(\log n)^{\mathcal{O}(1/\varepsilon)})$. The PTAS in [25] is extendible to dimension d with runtime $\mathcal{O}\left(n(\log n)^{\mathcal{O}(\sqrt{d}/\varepsilon)^{d-1}}\right)$. This PTAS was improved to $2^{(d/\varepsilon)^{\mathcal{O}(d)}} n + (d/\varepsilon)^{\mathcal{O}(d)} n \log n$ in [27,28]. Hence, for a fixed dimension d and a fixed approximation ratio ε the runtime is $\mathcal{O}(n \log n)$, however, with a large constant factor. As further improvement, a linear PTAS with expected runtime $2^{(d/\varepsilon)^{\mathcal{O}(d)}} n$ with high probability is proven in [29]. The work in [30] showed that a PTAS is possible even for the metric TSP whenever we restrict to a fixed doubling dimension.

Besides these more theoretical methods, there exists a large variety of more practical heuristics that try to achieve several goals: being efficient, generate good solutions close to the optimum — at least for the instances of interest — , and guarantee a somewhat small approximation ratio.

The apparently best approach is the modified Lin–Kernighan-algorithm by Helsgaun [31,32] (current version 3.0.10) that has an experimental runtime of $\mathcal{O}(n^{2.2})$ but runs in many cases much faster even on large instances. As a sophisticated implementation of the Lin–Kernighan heuristic [33], it has an approximation ratio of $\Theta(\log n / \log\log n)$ for the Euclidean TSP (claimed in [34]). On many benchmark suites, this LKH-algorithm achieves solutions very close to the optimum (gap well below 1% in many cases).

There are other heuristics that show better approximation ratios in the worst case, such as the cheapest insertion algorithm, which starts at an arbitrary point and expands the tour by inserting as next point the one with least increase in tour length. Cheapest insertion achieves an approximation ratio of 2 [35], however, the approximation to the optimal tour length is usually close to a gap of 20%. Heuristic algorithms from the field of evolutionary computation perform well solving the traveling salesman problem but only on small problem instances due to their memory requirements and runtime, see for instance [36]. Even in combination with improving algorithms such as LKH the runtimes become usually unhandable whenever the number of points exceeds several thousand points [37]. For that reason, we will not deal with this type of algorithms here in detail, besides the evidences added in Section 3.4.

We present a new algorithm as a heuristic to find a good approximation quickly in case of the Euclidean TSP, called *pair-center algorithm*, that uses a novel hierarchical clustering of the points and eventually achieves a deterministic worst-case linearithmic runtime, i.e., $\mathcal{O}(n \log n)$. A more practical implementation of the principal idea has a worst-case quasi-linear runtime, i.e., $\mathcal{O}(n \operatorname{polylog}(n))$. The measured runtimes on the experiments that we have run over benchmark instances up to many million points are approximately linearithmic. The average gap of the tours generated by the pair-center algorithm are below 5% on a variety of benchmark suites and approaches 0% on many of the smaller instances.

The rest of the article is structured in the following way. Section 2 describes the pair-center algorithm, Section 3 compares extensively our practical implementation with other heuristic approaches having a runtime less than or close to $\mathcal{O}(n^2)$, Section 4 provides runtime measurements to demonstrate the dependence of the runtime according to n and other free parameters of the algorithm. Section 5 summarizes the principal properties of the pair-center algorithm and proposes work still to be done.

2. The pair-center algorithm

Given a set of points in the Euclidean plane, the pair-center algorithm runs in two phases: a clustering phase and a construction phase.

In the first phase, the clustering phase, a binary tree over the points is built by successively substituting the currently closest pair of points by its center point. Eventually, only one center point remains as the root of the tree. The leaves of the tree contain the original points. The inner nodes of the tree contain the constructed center points. We store at each center point the distance between the two points of the underlying pair. This binary tree can be viewed as a hierarchical clustering of the point set where each subtree of the binary tree represents a cluster of the points at its leaves.

In the second phase, the construction phase, a traversal of the binary tree takes place that eventually constructs the complete tour. The traversal of the tree starts at the root of the binary tree that has been built in the first phase. A point of an inner node belonging to the current tour is replaced by its two generating points. Eventually, all inner nodes will have been replaced and the tour passes through all original points. Note that in this construction phase, there is always a closed tour available.

In the following, we describe the two phases of the pair-center algorithm in more detail. We give an example of a run with a 9 point problem instance and we analyze the complexity of the algorithm. Let us start with briefly introducing the data structures that will be employed later.

2.1. Data structures

Besides common simple data structures such as *lists*, *queues*, and *arrays*, the pair-center algorithms uses *heaps*, *dynamic trees*, and — in the practical implementation given later — *geometrical search structures*.

A *heap* of n elements is a data structure of size $\mathcal{O}(n)$ that maintains a smallest element (according to a given total comparison function), and allows for efficient insertion and deletion operations. Given a heap of n elements, a minimal element can be retrieved in constant time. Removing the smallest element and inserting a new element can be done in time $\mathcal{O}(\log n)$. Instead of building a heap for the smallest element, say a *min-heap*, it is easy to build a heap for the largest element, say a *max-heap*, by just modifying the comparison function.

Given a set P of n d-dimensional points, the variant of a *dynamic tree* as described in [38] is a data structure of size $\mathcal{O}(n)$ that maintains a closest pair of P in $\mathcal{O}(\log n)$ time per insertion and deletion. In the algebraic computation tree model, this data structure is optimal up to a constant factor. Here, we deal with $d = 2$, but note that the constant factor in the runtime of the operations on a dynamic tree grows exponentially in d.

A *geometrical search structure* is a data structure that holds n geometrical objects (in our case 2- or 3-dimensional points) and allows for efficient operations, such as nearest neighbor queries, range queries, object insertion, and object deletion. There exists a variety of specific geometrical search structures in the literature, each providing different mixtures regarding the complexities of the different operations on the data structure. The best known complexity of fully dynamic geometrical search structures [39] are $\mathcal{O}(\log^2 n / \log\log n + k)$ worst case k-nearest-neighbor query time, $\mathcal{O}(\log^{3+\epsilon} n / \log\log n)$ amortized insertion time, and $\mathcal{O}(\log^{5+\epsilon} n / \log\log n)$ worst case deletion time, i.e., the operations show an $\mathcal{O}(\operatorname{polylog}(n))$ runtime. Typical examples of such geometrical search structures are *kd-trees*, *quadtrees*, *R-trees*, *B-trees*, etc. For our practical implementation of the pair-center algorithm (see Section 2.5 and especially Section 2.5.1) that eventually achieves quite good short tours, we will use the *R-tree* data structure with the R*-strategy when rebalancing the tree during update operations [40].

2.2. Description of the pair-center algorithm

We assume that there are no duplicate points. We distinguish between a theoretical description of the pair-center algorithm as a heuristic to solve the Euclidean traveling salesman problem and — in a later section — a more practical version that modifies this algorithm both in the first and in the second phase. The theoretical description uses as replacement strategy for the center points in the construction phase a basic one.

Let P be a given set of n points in the Euclidean plane. Store all the points of P as the leaves of a binary tree B (still without inner nodes). Build a dynamic tree T of all points in P. The two phases of the pair-center algorithms are:

1. Complete the binary tree B in the following way.
   While there is still a closest pair in T:
   (a) Take the closest pair.
   (b) Delete the two corresponding points, say p_i and p_j, from the dynamic tree T.
   (c) Calculate the center point c_{ij} of the two points p_i and p_j, i.e., $c_{ij} = 0.5 \cdot (p_i + p_j)$.
   (d) Insert c_{ij} into the dynamic tree T.
   (e) Insert c_{ij} as an inner node into the binary tree B with p_i and p_j as child nodes.

   Eventually, the dynamic tree T is reduced to just one point and the binary tree B contains $2n-1$ nodes, namely, $n-1$ inner nodes as the pair-centers and n leaves as points of P.
2. Traverse the binary tree and construct the tour by substituting points in the current tour by their generating pairs of center points. The order of substitution takes place according to a *max-heap* that stores the distances of the corresponding tree nodes and the corresponding indices, i.e., the *max-heap* stores entries of the form (d_{ij}, i, j, k) where k is the index of the center point. The entries are ordered according to the distance value (first component of the tuples); ties are broken according to a lexicographic comparison of the three indices (i, j, k). In other words, first construct a one-point tour with the center point that is stored at the root node of the binary tree B and insert the first item into the *max-heap*.
   While the *max-heap* is not empty, proceed:
   (a) Take the maximal entry of the *max-heap*, say point q, which can be retrieved with the index k. Let q_p and q_n be the previous and the next point of q on the tour being constructed so far.
   (b) Remove the maximal entry from the *max-heap*.
   (c) Check which option gives the lowest tour increase when replacing the tour segment $[q_p - q - q_n]$ either with $[q_p - q_i - q_j - q_n]$ or with $[q_p - q_j - q_i - q_n]$, where q_i and q_j are the underlying points that generated the center point.
   (d) Increase the tour by the better of the two options.
   (e) Whenever q_i and q_j are center points, insert the corresponding information into the *max-heap*.

   Eventually the *max-heap* is empty and the final tour has been constructed.

Observe that, because we replace a closest pair of points by its center point, it cannot happen that a center point coincides with any other point of the point set.

2.3. Example invocation of the algorithm

Figs. 1 and 2 illustrate the application of the pair-center algorithm to an instance of 9 points (black dots). Fig. 1 shows the clustering phase of the algorithm. The plots demonstrate how the binary tree is constructed by consecutively adding new center points (reddish points) as new inner nodes of the tree. Eventually, a complete binary tree is obtained (green tree-like structure).

Fig. 2 shows the construction phase of the algorithm. The 9 plots demonstrate the iterative construction of the tour where in each step one additional point is added to the tour, i.e., one center point is removed and the two underlying points are added. The tour starts with just one point, the root of the binary tree from the clustering phase (upper-left plot, reddish point). The tour is enlarged by replacing iteratively center points. The last plot in the series shows the final tour.

2.4. Time complexity of the pair-center algorithm

The asymptotic time complexity of the theoretical pair-center algorithm is $\mathcal{O}(n \log n)$ where n is the number of points in the point set, which can be seen as follows:

1. Building the initial (only-leaves) binary tree takes time $\mathcal{O}(n)$.
2. Building a dynamic tree data structure takes time $\mathcal{O}(n \log n)$.
3. (a) Finding a closest neighbor in a dynamic tree of size n takes constant time $\mathcal{O}(1)$.
   (b) Deleting a point from the dynamic tree takes time $\mathcal{O}(\log n)$.
   (c) Calculating the center points takes constant time $\mathcal{O}(1)$.
   (d) Inserting a new entry in the dynamic tree takes time $\mathcal{O}(\log n)$
   (e) Inserting an inner node into the binary tree takes constant time $\mathcal{O}(1)$.

   Eventually, the runtime of the loop in this step takes time $\mathcal{O}(n \log n)$.
4. Constructing a one-point tour and a one-entry *max-heap* takes constant time $\mathcal{O}(1)$. The *max-heap* will have at most $n-1$ entries as at most $n-1$ inner nodes are handled.
   (a) Looking up the maximal entry in a *max-heap* and locating the previous and next point on the tour takes constant time $\mathcal{O}(1)$.
   (b) Removing the maximal entry in the *max-heap* takes time $\mathcal{O}(\log n)$.
   (c) Checking which option gives the lowest tour increase takes constant time $\mathcal{O}(1)$.
   (d) Increasing the tour by removing one point and adding two takes constant time $\mathcal{O}(1)$.
   (e) Inserting a new item into the *max-heap* takes time $\mathcal{O}(\log n)$ (or just constant $\mathcal{O}(1)$ if a Fibonacci heap is used).

   Eventually, the runtime of the loop in this step takes time $\mathcal{O}(n \log n)$.
5. Reporting the final tour can be done in time $\mathcal{O}(n)$.

Eventually, the overall worst-case runtime of the pair-center algorithm is $\mathcal{O}(n \log n)$.

2.5. Practical implementation of the pair-center algorithm

We implemented the pair-center algorithm in C++ with data structures from the standard library. However, we did not implement a dynamic tree as suggested in [38]. Rather we take advantage of an *R-tree* which can report the closest neighbors quite efficiently. As *R-tree* we use the geometric index structure from the Boost library [41] where the *R-tree* uses the R*-strategy when rebalancing the tree during update operations. In the current implementation, where we did not spend any special effort to reduce space requirements, roughly 300 bytes are needed per point. On our 128 GB test machine, problems with roughly 400 000 000 points can be handled.

In the first phase, we build a complete balanced *R-tree* data structure that contains all points. With the help of this *R-tree*, for each point p_i, we search its closest neighbor, say p_j. We introduce the

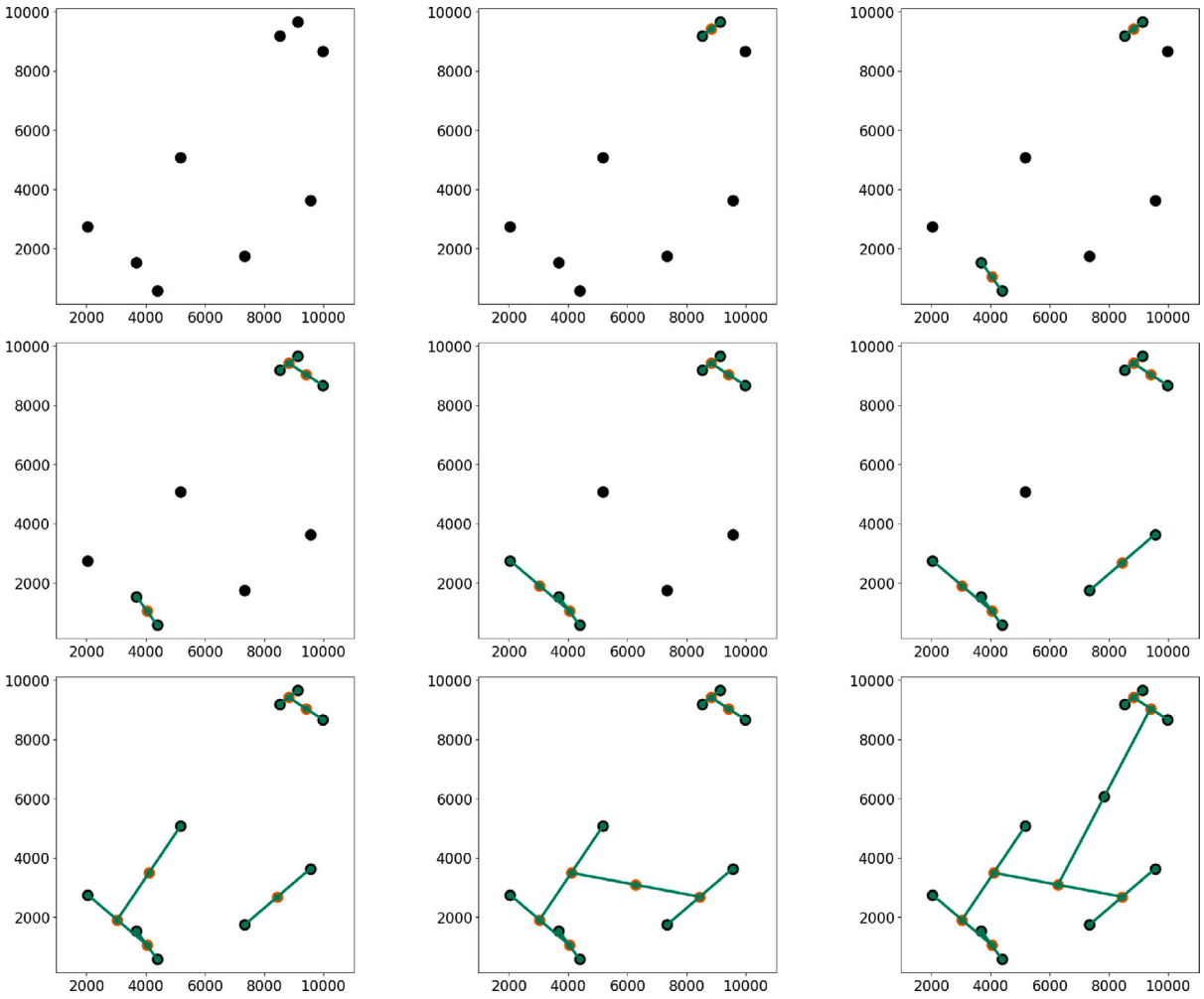

Fig. 1. Visualization of the first phase — the clustering phase — of the pair-center algorithm.

distances to the neighbors, i.e., $d_{ij} = d(p_i, p_j)$ into a *min-heap* with the corresponding pairs of points as payloads, i.e., the *min-heap* contains tuples like (d_{ij}, i, j), the same way as done in the theoretical algorithm. Additionally, we maintain for each point p_i the list of points that have this point as closest neighbor. We call the list of p_i its neighbor list $N(p_i)$. When we build the binary tree over the points in P we proceed as follows.

While the *min-heap* is not empty:

1. Take the minimal entry of the *min-heap*, reject those entries where not both points are still present, otherwise:
2. calculate the center point c_{ij} of the corresponding pair of points, say p_i and p_j, insert c_{ij} into the *R-tree*, remove the points p_i and p_j from the *R-tree*, and update the neighbor lists of the neighbors of p_i and p_j, i.e., the neighbor lists $N(q_k)$ for $q_k \in N(p_i)$ and $N(q_k)$ for $q_k \in N(p_j)$, and finally compute the new neighbor list $N(c_{ij})$,
3. insert c_{ij} as an inner node into the binary tree B, and
4. remove the minimal entry and insert the newly found closest pairs into the *min-heap*.

Observe that the number of closest points to a given point is confined by a constant value, namely the kissing number—which is 6 in the Euclidean plane. When deleting a point, only its neighbors may require an update.

The second phase runs as described in the theoretical algorithm as given above. We call this practical implementation that uses an *R-tree* data structure the basic pair-center algorithm, PCb. Employing the dynamic geometrical search structure to explore the close neighborhood of the points, we obtain a worst case $\mathcal{O}(n \operatorname{polylog}(n))$ runtime. However, in Section 4 we will see that this practical implementation on the base of an *R-tree* maintains an experimental runtime which lies in the order of $n \log n$ over very diverse benchmark suites.

In the ongoing, we introduce different variants of the pair-center algorithm that eventually achieve low gap values without increasing the overall runtime more than by a constant factor.

2.5.1. Neighbor search in the construction phase

Observe, that we can modify the replacement of the center point in the construction phase and implement it as a deletion of the center point and an insertion of both generating points q_i and q_j in the sense of a *locally best-detectable* modification of the tour which means that we can search in the neighborhood of the points q_i and q_j to find a tour point (and hence a tour segment) of the currently constructed tour where the new point can be inserted with an overall low increase in length. Therefore, we start in the second phase with an empty *R-tree* and insert the newly constructed tour points into this *R-tree*. Center points that are replaced are deleted from the *R-tree*. Performing a range query with a distance $f \cdot d(q_i, q_j)$ or performing a k-nearest-neighbor query are possibilities to obtain segments where to insert a point into the tour. Increasing the search distance or increasing the number k of neighbors decreases the final tour length as it will be shown in Section 4.

2.5.2. Different binary tree traversal

The basic pair-center algorithm takes in the construction phase as next center point the one with largest distance from any other point of

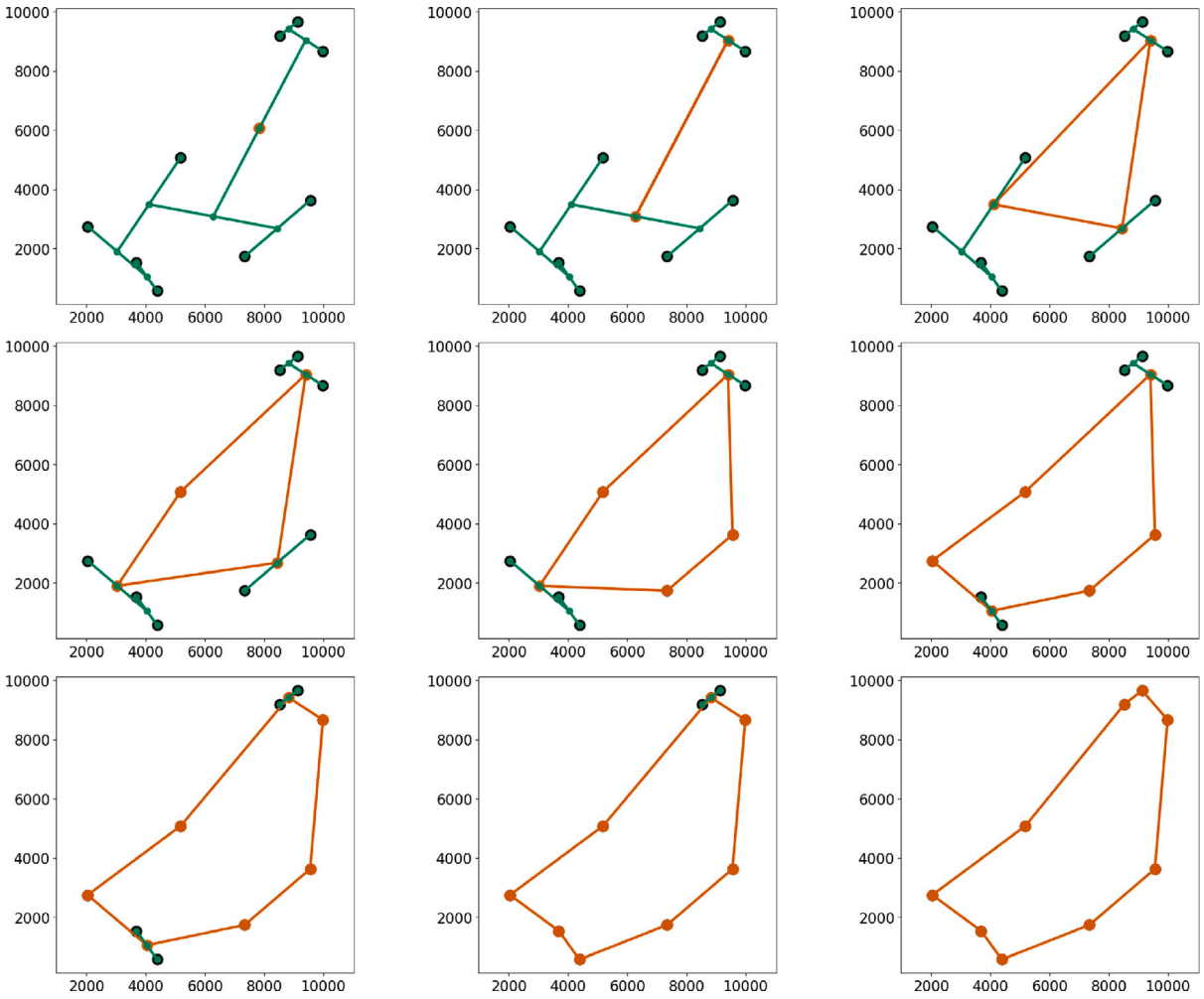

Fig. 2. Visualization of the second phase — the construction phase — of the pair-center algorithm.

the current tour (recall the *max-heap* data structure employed). Instead of the *max-heap* we can use either a *queue* or a *stack*, i.e., the traversal of the binary tree becomes a breadth first search or a depth first search of the binary tree, respectively. Note that, especially when using a *stack* and a range query for the closest neighbors the overall runtime of the algorithm might increase, because the *R-tree* update operations might become more costly. With such a modified traversal of the binary tree, sometimes a better tour is found (and sometimes a worse one, as expected for a heuristic).

2.5.3. Optimization of intermediate tours

The pair-center algorithm maintains in the construction phase always a closed tour and the replacement of a center point is done locally. Hence, we can try to improve the intermediate tours with any suitable tour improvement algorithm without changing the overall algorithm besides the fact that maybe a shorter tour is found. If we apply tour improvements with a runtime of at most $\mathcal{O}(n \log n)$ at intermediate tours at steps with exponentially increasing number of points, e.g., with 4, 8, 16, 32, etc. points, the overall algorithm remains in the order of $\mathcal{O}(n \log n)$ as well. Obviously, we can always add a suitable tour improvement phase on the final tour of size n.

2.5.4. Deterministic algorithm

In principal, the pair-center algorithm is deterministic, however, breaking the ties when distances between points are identical may produce different tours whenever the input sequence of points is given in a different order. If a fully deterministic algorithm is needed, we can sort in a preprocessing step the points lexicographically according to their x– and y–coordinates and index the points according to the sort order. That way, a unique list of points $P = (p_1, \ldots, p_n)$ is obtained. The sort order can be used in the comparisons to break the ties deterministically. Note that, in general, the order in which ties are broken might have a certain influence on the tour length that eventually is achieved.

2.5.5. Randomization

Although initially designed as a deterministic algorithm, we can randomize the pair-center algorithm in order to explore the solution space with the aim to find a better overall tour. A simple way to add randomization consists in jittering the center points in the clustering phase. In other words, instead of calculating $c_{ij} = 0.5 \cdot (p_i + p_j)$, we compute $c_{ij} = (0.5 + r) \cdot p_i + (0.5 - r) \cdot p_j$ where r is a random value taken, for instance, from a Gaussian distribution with zero mean and suitable standard deviation. Note that this randomization does not add any additional steps to the algorithm — besides running the algorithm a certain number of times —, i.e., the randomized pair-center algorithm still has complexity $\mathcal{O}(n \operatorname{polylog}(n))$.

2.5.6. Open tours

For some applications the shortest open loop that runs through all points exactly once is of interest. The pair-center algorithm can be extended easily to deal with this case. The first phase need not to be changed at all. At the start of the second phase, we build a two-point initial tour where the first point is the center point stored at the root of the binary tree from the first phase and the second point is a virtual

Table 1
Set of test instances to benchmark the algorithms.

Name	Instances	Number of points		Distance	Reference
tsplib	84	16	85 900	integral	[44]
nations	28	29	71 009	integral	[45]
vlsi	102	131	744 710	integral	[45]
dimacsC	23	100	316 200	integral	[46]
dimacsE	26	100	10 000 000	integral	[46]
arts	6	120 000	200 000	integral	[45]
santa	1		1 437 195	Euclidean	[47,48]
rand	19	100	100 000 000	Euclidean	own
hard	50	52	199	Euclidean	[20]
dots	6 449	6	31	Euclidean	[49–51]

point with no coordinates. As we need in the progress of the algorithm only the *increases* in tour length when we add a point, the distances to the virtual point can be assumed to be zero, i.e., in the construction phase of the algorithm the distances to the points q_p or q_n are set to zero whenever any one of the two points happens to be the virtual point. To obtain the final open tour, the virtual point is simply omitted. Note that sometimes the closed tour without its largest segment might be shorter than the one found by the open loop pair-center algorithm. In Section 3.3 we give some benchmark results on open tours.

2.5.7. 3D tours

The basic pair-center algorithm uses only distances between pairs of points, hence, the algorithm is trivial to be extended to higher dimensions. Note that the dynamic tree [38] maintains worst case $\mathcal{O}(\log n)$ complexity for all operations on the data structure, although the constants increase exponentially with the dimension. Therefore, the pair-center algorithm keeps running in time $\mathcal{O}(n \log n)$ assuming a fixed dimension d. Using a geometric search structure such as the *R-tree*, might increase the complexity of the algorithm with a higher exponent in the logarithmic factor. In Section 3.5 we give some experimental evidences for 3D tours.

3. Benchmark results

To compare two algorithms we use the gap defined as

$$gap = 100 \cdot (L - L_{\text{low}})/L_{\text{low}}$$

where L is the length of the tour generated by the corresponding algorithm and L_{low} is a lower bound. Whenever the optimal tour length is not known, we use either the Held–Karp lower bound [42] or some other empirical lower bound, e.g., as given in the benchmark suites, or as estimated for uniformly distributed points in a square $L_{low} = 0.7124\sqrt{n}$, see [43]. Note that a gap might be arbitrary large: arranging the points equidistantly on a circle with radius r, for increasing n the optimal tour length approaches $2\pi r$, whereas we can construct a tour of length almost $2rn$ when we connect as next point on the tour always the farthest point (usually right at the opposite side of the circle). Hence, the approximation ratio becomes close to n/π.

To allow for a more detailed comparison of two heuristics, besides the average gap on all instances of a certain benchmark suite, we use a tournament comparison, i.e., how often the pair-center algorithm wins (↑) or looses (↓) regarding the gap on the problem instances. Additionally, we plot the gap values versus the problem sizes for all algorithms.

3.1. The benchmark suites

We use a large set of test instances (see Table 1) to benchmark the pair-center algorithm and to compare it to other recent approaches. We removed all duplicates of points in the files. The suites *tsplib*, *arts*, *nations*, *vlsi*, *dimacsC*, and *dimacsE* use the tsplib standard way to compute the tour length with integral inter-point distances, whereas the suites *hard*, *dots*, *rand*, and *santa* use directly the Euclidean distance.

3.2. The algorithms to compare to

We compare the pair-center algorithm to the following nine algorithms being published in recent years that have a runtime close to $O(n^2)$: a polynomial-time deterministic approach (we will call it PTD) from [52], the distance matrix based approach MVODM from [53], the construction-insertion-improvement (CII) approach from [54], the fast popmusic algorithm from [55], the DoLoWire heuristic from [56] with its further improvement named Sys2To6 in [57], the GLNS approach developed for the generalized TSP [11], and the 2-opt++ approach recently published in [58]. Finally, we add a comparison to an algorithm from the field of evolutionary computation [36]. We have not included algorithms based on space-filling curves (such as the one in [59]) as they have a large gap, usually greater than 15%, and are not easily extendable to higher dimensions. For all algorithms that use randomization, we show the results for the best run we observed in the experiments.

PTD: The work in [52] describes an algorithm that first computes for each point a priority value according to a power function that depends on the mean distance and the standard deviation of the distances to all other points and then builds the tour by joining nodes with highest priority. As a final step, a local search based on 2-opt operations is applied. The paper states a polynomial deterministic runtime of $\mathcal{O}(n^2)$, however, the arguments are unclear, the pseudo-code provided exhibits a runtime of $\mathcal{O}(n^2 \log n)$.

MVODM: The work in [53] presents a distance matrix based algorithm for solving the traveling salesman problem. It significantly improves over the classical edge greedy algorithm that constructs a tour by selecting the next edge to add to the tour among the shortest edges available. The improvement is achieved by taking into account the variance of the distance matrix when selecting the edges that should participate in the tour, which reduces the chance that very large edges must be added in the final stages of the algorithm. The algorithm has time complexity $\mathcal{O}(n^2)$ but they present a heuristic approximation that uses an empirically adjusted parameter (named α in the range of 50–70) and an estimation of the variance considering only the distances to some central point. Eventually this approximation algorithm achieves a runtime in the order of $\mathcal{O}(n \log n)$.

CII: The heuristic described in [54] uses a three phase approach. In the first phase a convex hull polygon is computed in $\mathcal{O}(n^2)$ time that forms the initial tour. (Note, there are algorithms available that compute the convex hull in $\mathcal{O}(n \log n)$ time.) In the second phase the rest of the points is inserted choosing in each step the one that produces the least increase in tour length. It is stated that the second step runs in time $\mathcal{O}(n^2)$. In the third phase, the tour as generated in step 2 is improved with a local search strategy based on 2-opt operations.

popmusic: The popmusic approach as described in [55] is a heuristic, randomized algorithm with essentially only one parameter, named t, that runs in $\mathcal{O}(n \log n)$ expected time (assuming a fixed value of t). The initial tour is generated with the help of a recursive procedure. First t random points are selected and — with a suitable good tour construction algorithm — a tour of these t points is constructed. The rest of the points is distributed among the t points according to the Voronoi regions of the t points they belong too. In each region the procedure is run recursively taking care to build eventually a closed tour. In a second phase, this initial tour is divided into paths of length t^2 points starting at some random point, say q. Each path is improved with a suitable optimization algorithm. This procedure is repeated again with paths of length t^2, but starting now at a point being $t^2/2$ points away from q along the tour. Note that in [55] both for the construction algorithm as well as for the optimization algorithm, the Lin–Kernighan method is used. As said in Section 2.5.3, we use this second phase of popmusic to improve the intermediate tours generated in the construction phase of the pair-center algorithm.

DoLoWire: The DoLoWire approach introduced in [48,56] was second ranked in the SantaClaus challenge (*santa* benchmark). The algorithm first establishes a certain number of clusters based on a suitable grid and builds an initial tour connecting the centroids of these clusters. The number of clusters is a free parameter of the algorithm. The initial tour is improved with 2- and 3-opt operations. Afterwards, the cluster-connecting bridges are built by the closest pair of points connecting two corresponding clusters. Now, the tours in the individual clusters are built with the same approach as the initial tour is generated.

Sys2To6: In [57] the initial tour generated with DoLoWire is improved with a systematic application of 2- to 6-opt operations. Additionally, a sophisticated use of parallel processing with up to 30 jobs is employed to reduce the runtime as much as possible.

2-opt++: A recent work [58] introduces a randomized algorithm stating that it improves over the classical 2-opt approach. As initial tour a random tour with the nearest-neighbor greedy method is taken. Then, this tour is improved iteratively with a certain loop of 2-opt operations where before each iteration the Hamilton cycle is shuffled by taking randomly two sets of three consecutive points that are shifted clockwise. As a greedy approach, only tour improving operations steps are eventually recorded in the tour, non-improving iterations are discarded. As free parameter the number of iterations must be set beforehand. The algorithm has $\mathcal{O}(n^2)$ space requirements. No formal runtime analysis is presented, however, the pseudo-code shows 4 nested for-loops which seems to lead at least to a runtime of at least $\mathcal{O}(n^2)$, whenever the free parameters are considered as constants.

GLNS: The GLNS approach [11], here briefly described from an eTSP point of view, i.e., each location is its own subset that must be visited, is a randomized approximation algorithm based on more or less standard simulated annealing. The annealing loop is additionally embedded in a multi-trial loop. During the simulation better tours are always excepted, worse tours are excepted according to the current temperature value. The process stops whenever stagnation is detected or a given runtime limit has been reached or a tour with given length has been found. To escape local minima, a fixed number of warm starts beginning with the best solution found so far is executed.

The algorithm starts either with a random tour or a nearest insertion tour. The inner step of the simulated annealing loop first removes a certain subset of nodes with different removal strategies and reinserts the subset with elegantly chosen randomized insertion strategies (based on nearest, cheapest, random, and furthest insertion). There is a small set of free parameters that are preset to certain values to generate different variants dealing with the trade-off between overall runtime and gap values. The slow variant finds better gaps but needs more time, the fast variant needs less time to converge (stagnate) but usually generate tours with larger gaps. We have customized some of the free parameters to produce a custom variant with properties inbetween slow and fast. GLNS is highly randomized with a selection mechanisms based on iteratively changing weights that determine how to choose which points to remove and in which order to reinsert. Due to the randomization, there are large runtime variations until stagnation (i.e., no further improvement) is detected, 50% difference can be observed when the same instance is handled. The current implementation needs $\mathcal{O}(n^2)$ memory space.

EVO: Evolutionary algorithms (also called bio-inspired or nature-inspired algorithms) perform well as heuristics to solve small sized traveling salesman problems. The recent work in [36] compares a grey wolf optimizer (GWO) to other twelve approaches from this field and, based on experiments, indicates that their TO-GWO algorithm shows a better performance compared to the others. We will not describe in detail any of these evolutionary algorithms (a task out of scope for this article), rather we just compare the results given in [36] to those of the pair-center algorithm. No runtime analysis nor runtime measurements are provided in [36]. Inspecting the pseudo-code they provide, we can derive that at least $\mathcal{O}(sn)$ space is used, where s is the number of individuals used in the population based algorithm ($s = 100$ for their experiments). The same way, we can derive that at least $\mathcal{O}(tn)$ time is used, where t is the number of iterations used in the evolutionary algorithm ($t = 1\,000$ for their experiments). So whenever t is chosen larger than $\log n$, TO-GWO will be slower that the pair-center algorithm and, probably, the population size must be chosen small to compete in memory requirements.

3.3. The gap of different variants of the pair-center algorithm

We show the results for the following variants of the pair-center algorithm: PCb, the basic pair-center algorithm as described in Section 2.5; PCk, the pair-center algorithm with a k-nearest-neighbor search as described in Section 2.5.1 using always $k = 26$ (which will be motivated in Section 4); PC, the best results of the four variants using a k-nearest-neighbor search with always $k = 26$ or a range query with always $f = 7$ (see Section 2.5.1) combined with either a *max-heap* or a simple *queue* in the construction phase (see Section 2.5.2); three further variants that use the second phase of the popmusic approach in the construction phase as described in Section 2.5.3; and, finally, four more variants, namely both the PCk and the PC variant with additional randomization in the clustering phase as described in Section 2.5.5, once with 10 runs and once with 100 runs. Eventually, we obtain the data in the ten columns of Table 2 where the average gap on all benchmark suites is shown. The zero in italics means that it is a rounded value, actually slightly above zero.

The basic pair-center algorithm (PCb) is already better than many other classical heuristics such as nearest addition, cheapest insertion, nearest insertion, farthest insertion, or random insertion on many data sets. For instance, on the *vlsi* suite these classical heuristics show an average gap close to 25% whereas PCb is below 20%. Note that these classical heuristics have runtimes of $\mathcal{O}(n^2)$ [60]. With the randomized variant of the pair-center algorithm using 100 runs, the optimal tour has been found 18 times for the instances of the *tsplib* suite and for the 2 smallest instances of the *nations* suite. One can observe that randomization on the instances of the *arts* suite does not decrease the

Table 2
Average gap values of different variants (see text for explanation) of the pair-center algorithm on the benchmark suites.

	PCb	PCk	PC	PCb	PCk	PC	PCk+10	PC+10	PCk+100	PC+100
				+popmusic						
				deterministic			+10 random		+100 random	
tsplib	16.11	6.28	5.97	4.03	3.82	3.46	2.90	2.62	2.55	2.31
nations	17.20	6.01	5.88	3.94	3.61	3.57	3.24	3.11	3.04	2.95
vlsi	19.66	8.99	8.87	5.92	5.71	5.55	5.10	4.95	4.76	4.62
dimacsC	17.20	6.12	6.00	3.87	3.57	3.53	3.15	3.01	2.85	2.76
dimacsE	15.95	7.39	7.37	5.24	4.90	4.87	4.50	4.44		
arts	12.03	4.76	4.76	2.95	2.82	2.74	2.79	2.71	2.78	2.70
santa	18.21	6.59	6.59	4.30	4.05	4.05	3.85	3.85		
rand	15.50	6.88	6.88	4.98	4.65	4.65				
hard	13.43	2.14	1.41	0.86	0.62	0.40	0.16	0.05	0.04	0.01
dots	1.28	0.81	0.52	0.19	0.16	0.08	0.01	*0.00*	*0.00*	0.00

gap significantly as it is the case in all other benchmark suites. The PC variant with popmusic and randomization finds for 31 problem instances of the *hard* suite the optimal tour. For all 6 449 instances of the *dots* suite, the variant of the pair-center algorithm with randomization finds all optimal tours, a result significantly better than the data in Table 2 of [50] where the best average is 0.01% and worst gap is 2.59%. For 5 of the 6 449 open loops, we found shorter tours as those given as ground truth in [50], but the very small error has been corrected in the online data.

3.4. Comparisons with the algorithms

Not all publications — we will compare to in the ongoing — consider the entire benchmark suites as listed in Section 3.1. Therefore, we add the number of problem instances for which data are given in the corresponding article in the third column of Table 3 which summarizes the gap values and the tournament comparisons for two variants of the pair-center algorithm, namely PCk and PC+10 (columns 6 and 9 from Table 2), and the algorithms described in Section 3.2. Boldface entries show the best (least) gap values or the cases where one algorithms wins on all instances for that benchmark. Italic entries show where the variant PCk is already better than the other algorithm. We have recalculated the average gap values for the variants of the pair-center algorithm for the same set of problem instances.

Fig. 3 shows the gap values versus instance sizes for all algorithms and benchmark suites compared to the pair-center algorithm variant PC+10 for all data available. The pair-center algorithm is visualized with yellowish dots, the other algorithms with reddish triangles. For 7 of the 9 algorithms, the pair-center shows better (smaller) gap values on average.

PTG: The gap is given for 20 instances of the *tsplib* suite with less than 500 points. PC+10 is clearly superior with only 0.37% average gap compared to 2.4% in [52] and eventually wins on all instances. Observe that the set of the five adjustable parameters in [52] is different for all instances which implies that at least 125 runs of the algorithm have been performed to achieve the best results as reported in the corresponding table in [52]. For the 20 instances, the approach never finds the optimal tour, whereas the pair-center algorithm finds the optimum for one instance using PCk and for five instances using PC+10.

MVODM: The gap in [53] is computed towards the Held–Karp bound. Therefore we have adapted the data achieved by the pair-center algorithm to compute the gap to the Held–Karp bound and to use the Euclidean length, rather the rounded inter-point distances as usually done in the benchmark suites.

Table 3 shows the data for the 33 instances of the *tsplib* suite with more than 1 000 points and for the complete *dimacsC* and *dimacsE* suites. The pair-center algorithm outperforms the MVODM approach clearly.

CII: The pair-center algorithm outperforms the CII approach almost always. CII is only better on some of the *tsplib* instances. Note that on the largest instance available in [54] with 744 710 points (`lrb744710` from the *vlsi* benchmark), CII was interrupted after 15 days of computation reaching a final gap of 15.9%, whereas the PCk variant of the pair-center algorithm achieves a gap of 5.45% in roughly 46 s.

popmusic: Popmusic can only handle the Euclidean distance measure and closed loops, therefore some problem instances are not considered and no comparison on the *dots* suite was possible. Only the variants of the pair-center algorithm that do not use the popmusic optimization should be considered for a fair comparison, it is not surprising that PCk and PC+100 are almost always better than popmusic on average.

The deterministic variant is already better on average than popmusic on all benchmark suites except three, namely the *tsplib*, *arts*, and *hard* suites. Popmusic behaves particularly bad on the *santa* instance. There, it is even worse than the basic pair-center algorithm PCb (recall 18.21% gap for PCb in Table 2). In general, the larger the number of points the worse is the gap of the popmusic approach. For instance, for the larger examples ($n \geq 10\,000$) of the *dimacsC* benchmark the pair-center algorithm wins the tournament comparison in all variants shown in Table 2; popmusic reaches a gap of almost 15%, whereas the pair-center variants exhibit quite a constant gap over all instances: 2.5%–4.5% for the PCk variant with intermediate tour improvement. On the *tsplib* instances popmusic wins the tournaments in one third of the cases, here again, almost always this happens on the smaller instances (less than 1 000 points).

On the same machine, a single run of popmusic is roughly three to six times faster than PCk depending on the benchmark suite (see Fig. 5). On the problem instance with 100 million points of the *rand* suite, one run of popmusic is roughly seven times faster than one run of the pair-center algorithm PCk, however, PCk almost halves the gap. More details on the runtime behavior of the pair-center algorithm are included in Section 4.

DoLoWire: Only for a few instances from the benchmark suites there are data available in [56,57]. The values of the gap differ from the ones published as we use the gap according to the same lower bound as in the other comparisons, rather than the best known length as done in the original work. PCk is almost always and PC+10 is always better than DoLoWire.

Sys2To6: The Sys2To6 algorithm in general shows a lower average gap than the pair-center algorithm. However, it is remarkable that on two of the three large instances of the *vlsi* suite, the pair-center algorithm finds better tours. No runtime analysis is given in [57] rather the free parameters are tuned such that

Table 3
Average gap values and tournament comparisons for all algorithms with two variants of the pair-center algorithm.

Benchmark	Algorithm	Instances	gap			Tournament			
				PCk	PC+10	PCk		PC+10	
tsplib	PTD	20	2.40	3.06	**0.37**	↑18	↓2	↑**20**	↓0
	MVODM	33	9.89	*6.80*	**5.51**	↑27	↓6	↑31	↓2
	CII	81	5.23	*3.66*	**2.41**	↑62	↓19	↑72	↓9
	popmusic	78	3.32	*6.30*	**2.56**	↑7	↓71	↑45	↓27
	DoLoWire	6	8.74	*5.64*	**3.70**	↑5	↓1	↑**6**	↓0
	2-opt++	66	4.17	*3.19*	**1.85**	↑51	↓15	↑62	↓4
	GLNS custom	75	3.57	*3.45*	**2.10**	↑41	↓34	↑67	↓8
	GLNS fast	75	4.27	*3.45*	**2.10**	↑28	↓45	↑52	↓17
	GLNS slow	75	2.88	3.45	**2.10**	↑25	↓47	↑29	↓34
	EVO[a]	46	**0.54**	2.30	1.01	↑0	↓**46**	↑14	↓32
nations	CII	27	8.16	*3.62*	**2.41**	↑**26**	↓0	↑**26**	↓0
	popmusic	28	6.87	*6.01*	**3.11**	↑20	↓7	↑25	↓1
	DoLoWire	1	9.70	*4.78*	**4.61**	↑**1**	↓0	↑**1**	↓0
	GLNS custom	14	5.69	*3.07*	**2.46**	↑**12**	↓0	↑**12**	↓0
	GLNS fast	14	5.98	*3.07*	**2.46**	↑11	↓1	↑12	↓2
	GLNS slow	14	6.89	*3.07*	**2.46**	↑11	↓1	↑11	↓1
vlsi	CII	102	8.62	*5.71*	**4.95**	↑101	↓1	↑**102**	↓0
	popmusic	102	9.13	*5.71*	**4.95**	↑53	↓49	↑99	↓3
	DoLoWire	3	17.00	*5.44*	**5.40**	↑**3**	↓0	↑**3**	↓0
	Sys2To6	3	7.20	*5.44*	**5.40**	↑2	↓1	↑2	↓1
	GLNS custom	70	7.22	*5.22*	**4.24**	↑65	↓5	↑69	↓1
	GLNS fast	70	7.22	*5.22*	**4.24**	↑59	↓11	↑68	↓2
	GLNS slow	70	8.81	*5.22*	**4.24**	↑61	↓9	↑60	↓10
dimacsC	MVODM	23	15.20	*3.57*	**3.01**	↑**23**	↓0	↑**23**	↓0
	popmusic	23	7.21	*6.12*	**3.01**	↑10	↓13	↑**23**	↓0
	DoLoWire	2	7.50	*4.08*	**4.17**	↑**2**	↓0	↑**2**	↓0
	Sys2To6	2	**3.22**	4.08	4.17	↑0	↓2	↑0	↓2
dimacsE	MVDOM	26	9.02	*4.90*	**4.44**	↑**26**	↓0	↑**26**	↓0
	popmusic	26	7.50	*4.90*	**4.44**	↑12	↓14	↑**26**	↓0
	DoLoWire	3	13.18	*5.43*	**5.44**	↑**3**	↓0	↑**3**	↓0
	Sys2To6	3	**3.32**	5.43	5.44	↑0	↓**3**	↑0	↓**3**
arts	CII	6	3.57	*2.82*	**2.71**	↑**6**	↓0	↑**6**	↓0
	popmusic	6	3.46	*2.82*	**2.71**	↑**6**	↓0	↑**6**	↓0
	DoLoWire	1	4.51	*3.10*	**2.99**	↑**1**	↓0	↑**1**	↓0
	Sys2To6	1	**0.71**	3.10	2.99	↑0	↓**1**	↑0	↓**1**
santa	popmusic	1	20.08	*4.05*	**3.85**	↑**1**	↓0	↑**1**	↓0
	DoLoWire	1	10.82	*4.05*	**3.85**	↑**1**	↓0	↑**1**	↓0
	Sys2To6	1	**3.69**	4.05	3.85	↑0	↓**1**	↑0	↓**1**
rand	popmusic	19	8.73	*4.65*		↑**19**	↓0		
hard	popmusic	50	0.23	0.62	**0.05**	↑12	↓38	↑40	↓10
	GLNS custom	50	**0.04**	0.62	0.05	↑2	↓48	↑15	↓28
	GLNS fast	50	**0.01**	0.62	0.05	↑1	↓49	↑5	↓38
	GLNS slow	50	**0.00**	0.62	0.05	↑0	↓**50**	↑0	↓**42**

[a] Means that Euclidean distance gap is computed towards known integral optimum.

the computation stops after 30 min of processing. It would be interesting to find out what gap could be achieved using the pair-center algorithm as initial tour generator for Sys2To6.

2-opt++: There was data available for the 66 small instances from the *tsplib* benchmark suite (two of the instances are larger ones that had to be preprocessed to subdivide the problem into tiles that are solved and then stitched together to generate a closed tour). The pair-center algorithm outperforms the 2-opt++ approach clearly. For the largest instance with runtime data given (namely instance `pr439`) 2-opt++ needed 11.63 s to find a tour with 1.83% gap, whereas the pair-center algorithm finds deterministically a tour with a gap of 1.27% in 0.009 s, i.e., almost 1 300 times faster. The hardware they run the benchmarks was possibly 20% slower than ours, nevertheless, even on the small problem, the pair-center algorithm is three orders of magnitude faster.

GLNS: The GLNS approach in general finds tours with smaller gaps for small instances (less the 200 points, e.g., the *hard* suite). For larger (but still medium size instances) the pair-center algorithm performs better. Fig. 3 shows the data for the fast variant of GLNS because the average gap value was the smallest. However, the slow variant typically performs better on the small instances, but significantly worse on the large instance given the runtime limit of 1200 s.

EVO: First of all note that Table 4 in [36] does not present the data in the standard way for comparing solutions regarding the *tsplib* benchmark suite. On the one hand side they use as optimal tour lengths the sums of the rounded inter-point distances, but on the other hand side they actually compute their tour lengths just as sums of the corresponding Euclidean distances. This is the reason why some of their relative errors became negative. Effectively, they just found the optimal tours in these cases. Further, for the first four smallest instances, they used the Euclidean distances computed between the points, however, these examples give points with geographical coordinates or with a special inter-point distance matrix. Nevertheless, it seems that — assuming the points have just coordinates in the plane — the four examples are solved to the optimum. This was also the case for the pair-center algorithm; we found — under the same assumptions — all optimal tours for these four small problems.

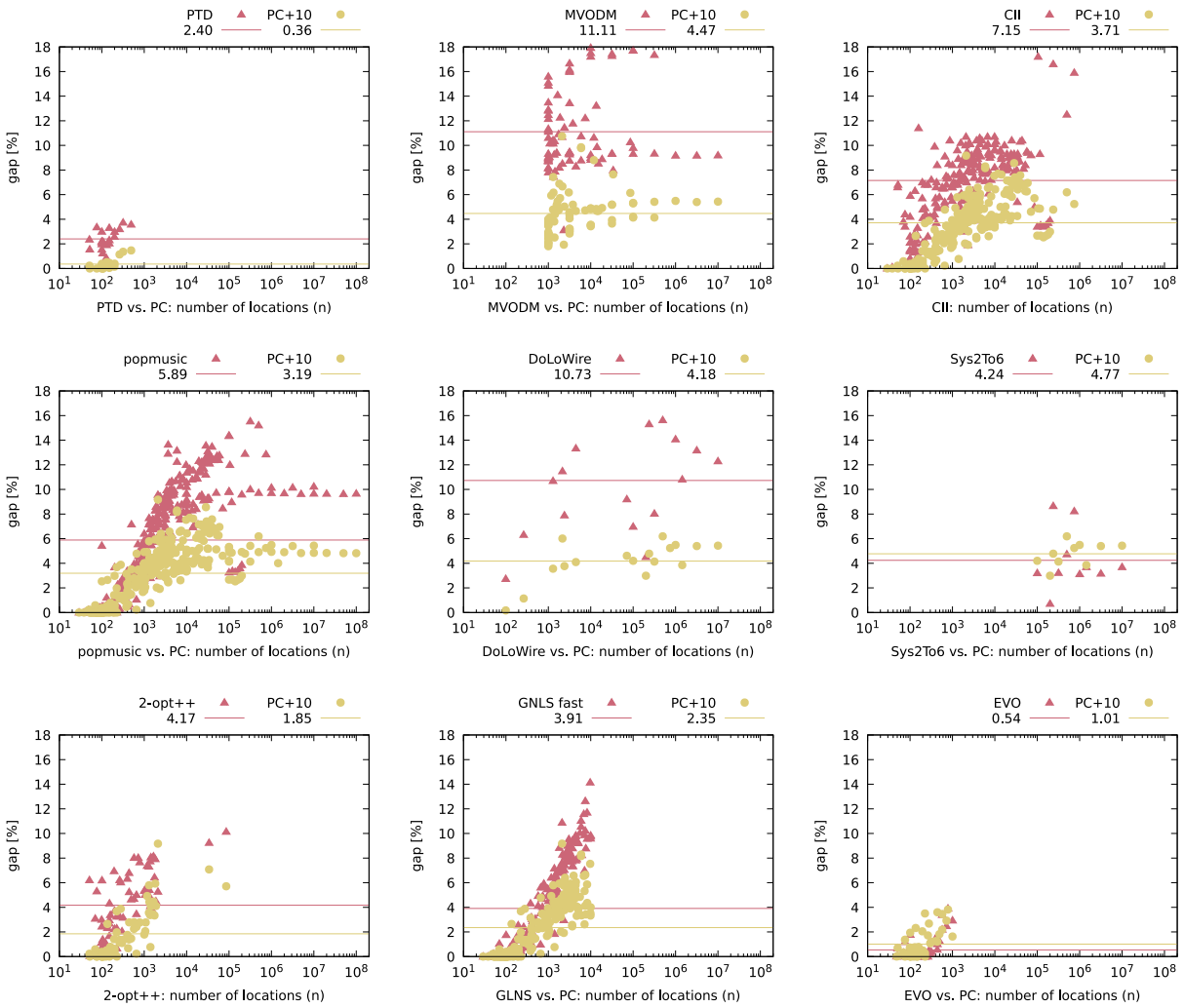

Fig. 3. Gap values versus point number for the different algorithms.

For the other 46 examples with up to 1 002 points from the *tsplib* suite, we compare the Euclidean lengths of their best results (column 4 in their Table 4), with the same quantities found by the pair-center algorithm. The average gap for TO-GWO is 0.54%, the pair-center algorithm reaches 1.01 (Table 3). If we consider only the five largest instances (those with more than 500 points), the pair-center algorithm always wins with a gap of 2.35%, while TO-GWO exhibits 2.98%. This fact highlights — as stated in the introduction — that evolutionary algorithms perform well on small instances, but due to memory and runtime limitations they behave worse on large instances; handling millions of points is out of reach [37].

3.5. Comparison on 3D data

Table 4 shows the gap of the pair-center algorithm with k-nearest-neighbor search and intermediate tour improvement and the gap of the popmusic approach on three-dimensional instances taken from the stars challenge [45]. No further changes have been applied to the code, besides using 3D coordinates (and a 3D *R-tree*). Both programs achieve gap values similar to those we observed for the 2D random instances. The runtime of PCk on the `gaia` instance with more the 2 million stars was 373 s, popmusic needed roughly 19 s for each of the 10 runs. However, the pair-center algorithm halves the gap on the larger instances investing twice the time.

Table 4
Gap values of the pair-center algorithm PCk with intermediate tour improvement compared to the popmusic approach (10 runs) on three-dimensional instances from the *stars* benchmark suite.

Instance	PCk	PCk+10	Popmusic
star1k	3.83	3.48	6.35
star10k	4.88	4.83	9.77
hyg109399	5.37	5.24	11.78
star250k	5.42	5.41	11.61
gaia2079471	5.36	5.37	12.30

4. Runtime measurements

We ran all experiments on a Linux-based platform with Intel processor. The system hosted a 10-core/20-thread Intel Core i9-7900X microprocessor with a maximum clock frequency of 3.3 GHz, 32/1024/1408 KiB 8/16/11-way-set-associative L1/L2/L3 caches, and 128 GB of memory. The maximum bandwidth to memory is approx. 85 GB/s. Note that only one core was active executing the single thread sequential pair-center algorithm. We implemented the pair-center algorithm in C++ with data structures from the standard library and from the Boost library [41] compiled with g++ version 11.4. All runtimes reflect wall-clock real time.

Fig. 4 shows on the left hand side the gap and on the right hand side the runtime of the PCk variant (k-nearest neighbor search) and the PCf variant (range query with factor f) for increasing values of

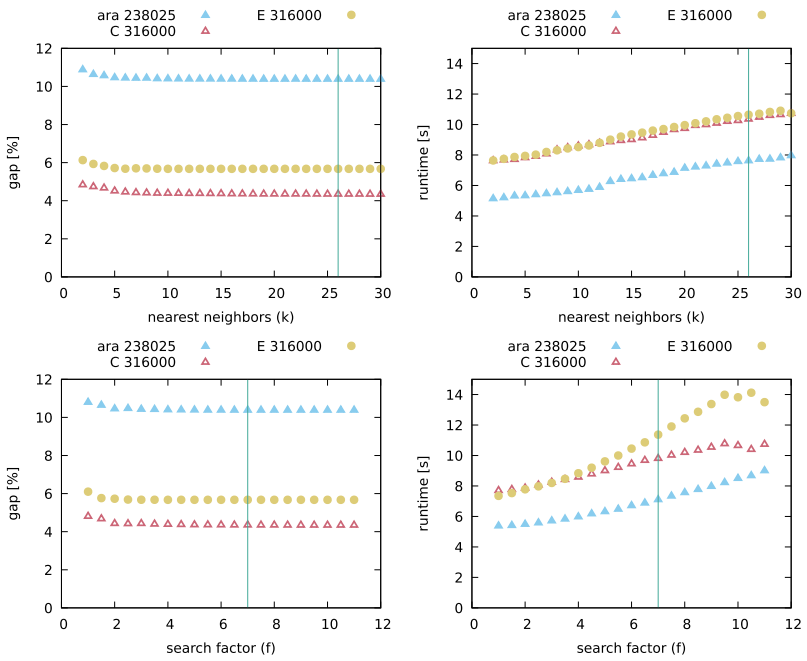

Fig. 4. Influence of the number of neighbors k for the neighbor search (upper plots) and the factor f for the range search (lower plots) on the gap and runtime for three large instances of the benchmark suites.

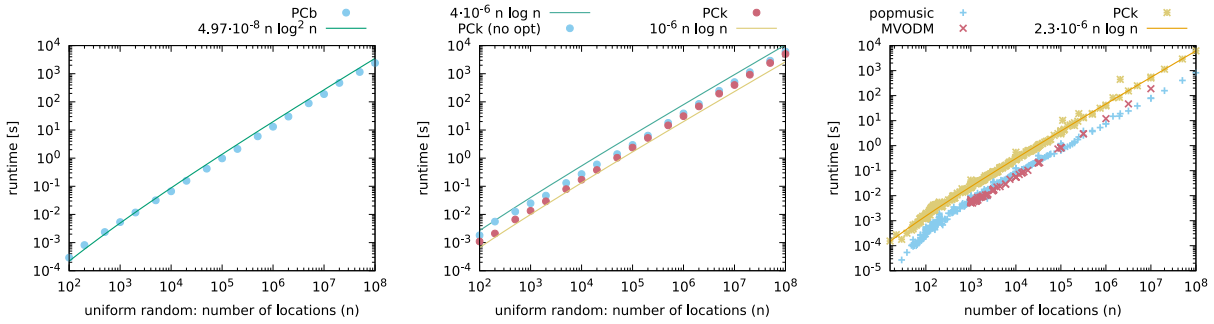

Fig. 5. Measured and fitted runtime of the basic pair-center algorithm (PCb) (left hand side) and measured runtimes for four different variants of the pair-center algorithm and their enclosing within $\mathcal{O}(n \log n)$ functions (right hand side).

k, respectively, f. We show the data for the clustered and the uniform distributions with 316 000 points from the *dimacs* suites and the `ara238025` example from the *vlsi* suite. Roughly with a value $k = 26$ no further significant improvement can be observed. This fact holds in general over all benchmark instances that we have tested. Similarly with a factor $f = 7$ no further significant improvement can be observed. In other words, setting either $k = 26$ or $f = 7$, we obtain parameter-free versions of the pair-center algorithm.

The basic pair-center algorithm (PCb) shows the measured runtimes on the examples of the uniform random distribution of points in a square (*rand* suite) as given in Fig. 5 (left hand side). The data has been fitted with the function $4.97 \cdot 10^{-8}\ n \log^2 n$ which demonstrates that the basic algorithm — using an *R-tree* as underlying geometric data structure — exhibits a quasi-linear runtime, at least on the uniformly distributed point sets.

Fig. 5 (center) shows the runtimes of two variants of the pair-center algorithm, namely, PCk (nearest-neighbor search with $k = 26$) both with and without intermediate tour improvement. We have enclosed the measured data with the functions $4 \cdot 10^{-6} \cdot n \log n$ (above) and $1 \cdot 10^{-6} \cdot n \log n$ (below) to visualize that the variants fit well in the interval between the two limits. Hence, at least in the experimental data, the variants are in the order of $n \log n$, with a very similar shape of the data plots. To re-confirm the estimation of the runtime, we plot the function $2.3 \cdot 10^{-6} \cdot n \log n$ against all measured runtimes of the pair-center algorithm with nearest neighbor search and intermediate tour improvement on the complete set of all instances available in the different benchmark suites (without considering the open loops of the *dots* suite), see Fig. 5 right hand side. In the plot, we have added the runtimes of the other two linearithmic algorithms, namely popmusic and MVODM.

Fig. 6 (left hand side) again shows the runtime of PCk but now separating the benchmark suites. The only points a little off the curve reflect the data for the *stars* suite. This is expected as this benchmark works with 3D locations. We have added on the right hand side of Fig. 6 all gap values for all benchmarks (again except for the *dots* suite as they are always zero). The average gap is 2.99% (not including the 6 449 instances of the *dots* suite).

As last comparison we have plotted in Fig. 7 for four instances the relation between gap value and runtime for all algorithms whenever there was data available. The examples have 4 461, 18 512, 100 000, and 238 025 points, respectively. One can clearly observe that the linearithmic approaches all lie very to the left in the logscale diagram indicating that they are orders of magnitude faster than the other methods. Regarding the gap values, the two variants of the pair-center algorithm shown in the plot are located in the lower left area, hence, exhibiting a good trade-off between runtime and gap. The other two linearithmic algorithms are faster but generate tours with larger gaps.

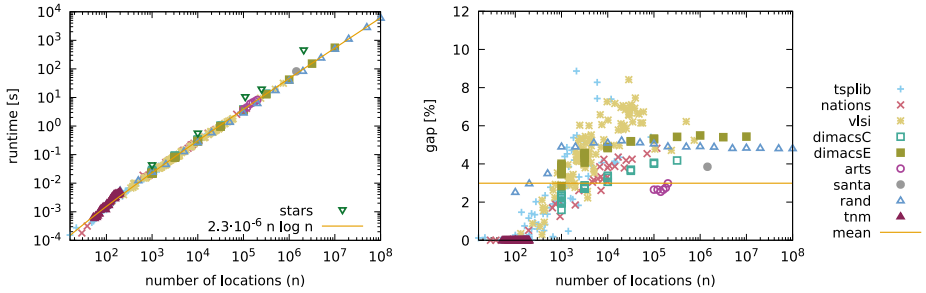

Fig. 6. Approximation of the measured runtime with an $\mathcal{O}(n \log n)$ estimation of the pair-center algorithm with nearest neighbor search and intermediate tour improvement (left hand side) and gap of all instances (right hand side) on all benchmark suites, but *dots*.

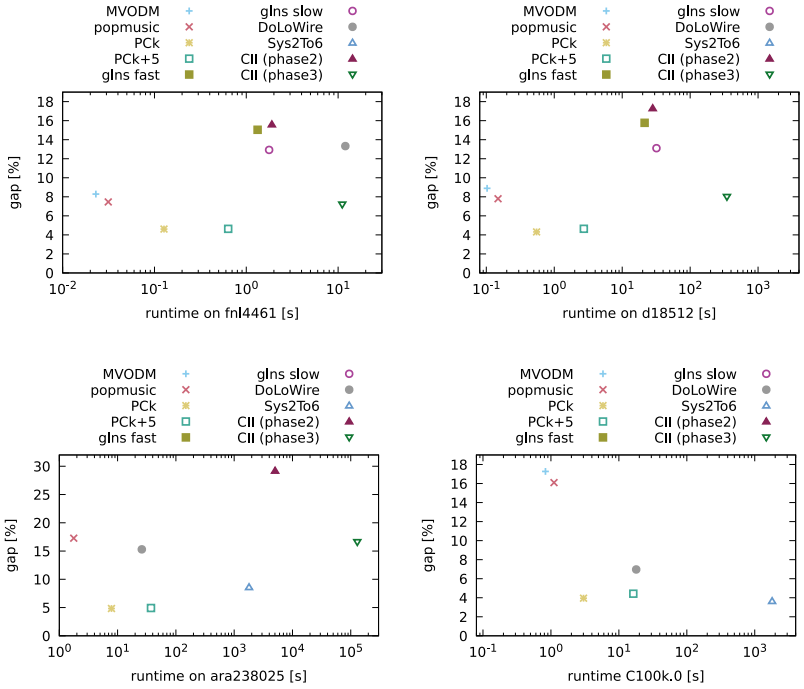

Fig. 7. Gap versus runtime.

The pair-center algorithm, with the exception of Sys2To6 on the 100k data set, shows always the best gap.

5. Conclusions and future work

The pair-center shows very consistent behavior regarding its quasi-linear runtime over a variety of benchmark suites with both a theoretical runtime of $O(n \log n)$ for the basic pair-center algorithms and a practical runtime of $O(n \log n)$ for the different variants implemented with the help of an *R-tree* data structure. On our test machine the runtime can be approximated very closely with the function $2.3 \cdot 10^{-6} \cdot n \log n$. The space requirements are linear in the number of points. The gap of the tours found by the pair-center algorithm is very small or even zero for smaller instances. The average gap of the deterministic variant over all benchmarks with closed tours is 3.92%. The average can be reduced to 3.35 with the variant that includes a 10-fold randomization. Taking the minimum over several variants (called PC in Table 2) with optimization and randomization the average gap is 2.99%. The average on all 148 instances with less than or equal to 1 000 points is 0.94%, for the 192 instances with more than 1 000 points the average gap is 4.57%. The gap is significant better than documented for any other previously published heuristic with at most $O(n^2)$ runtime. The algorithm generates only non-self-intersecting tours.

The LKH-algorithm [31,32] or the GLNS-algorithm [11] are still the best approximation heuristics to find very good solutions up to medium size TSP problems. However, whenever it comes to large instances, or a good solution must be found in a limited amount of time, the pair-center algorithm is a new and valuable alternative as documented in this article. Helsgaun [31] states that significant reduction in runtime may be achieved by choosing initial tours that are close to being optimal. So the pair-center algorithm might be another choice to start other approximation algorithms that improve the initial tour iteratively.

We have observed that randomization, at least the one we implemented, helps to find better tours for problem instances with small or median sized point sets. For large point sets, the randomization becomes costly (as a single run already takes its time), and shaves only a small fraction of the gap (and sometimes not even that). Here, maybe ideas that select good partial tours among a certain number of randomized runs that eventually are glued together in some way may lead to better tours, here, more research is needed.

As future work, we also want to extend our experiments in detecting certain types of clusters in the point sets. Maybe the principal idea of the pair-center algorithm can be used in other more general TSP scenarios as well. One might try to prove the approximation ratio of the pair-center algorithm. Is there a constant upper bound on the gap of the pair-center algorithm similar to nearest insertion or cheapest insertion, as it is essentially quite similar to a cheapest neighbor insertion (in the

construction phase) based on an approximate minimum spanning tree (built in the clustering phase)?

As outlook, we currently believe that — with the 3D pair-center algorithm — a not so bad approximation for the 1.33 billion stars challenge [45] can be found within roughly one day on a single processor machine with 1 TB of main memory.

Regarding the implementation, the *R-tree* data structure in the Boost library degrades significantly when the point set is distributed in linear clusters (even more whenever these clusters run in directions of the coordinate axis). This effect should be handled, if possible, in future work on geometrical search structures. The pair-center algorithm poses a special case for the geometrical search structures such as the *R-tree*: all initial points are known and in the two phases they are treated in a certain order: from shorter to larger distances, and vice versa. With this additional knowledge, maybe the practical complexity of the *R-tree* data structure can be reduced. Further, a more specific implementation needs to be done, for instance, storing only the indices into the point set instead of the coordinates, which would decrease the memory footprint of the implementation significantly. One might experiment with other types of dynamic geometric search structures, as well.

Currently, we do not use any parallelization in the algorithm. However, there is some potential, for instance, searching within the nearest neighbors for the point with best insertion value can be done in parallel and the optimization of short partial tours in the popmusic intermediate optimization is easy to parallelize.

Contribution

All work for this article has been done by the author. No artificial intelligence tools have been used.

CRediT authorship contribution statement

Arno Formella: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

We like to thank Eric Taillard, Stephen Smith, and Frank Imeson who give access to the source code of their implementations and Pasi Fränti for early discussions on the pair-center algorithm.

References

[1] G. Gutin, A.P. Punnen (Eds.), The Traveling Salesman Problem and Its Variations, Springer, Boston, MA, USA, 2007.
[2] D.L. Applegate, R.E. Bixby, V. Chvátal, W.J. Cook, The Traveling Salesman Problem: A Computational Study, Princeton University Press, 2011.
[3] R. Roberti, P. Toth, Models and algorithms for the asymmetric traveling salesman problem: an experimental comparison, EURO J. Transp. Logist. 1 (1) (2012) 113–133, http://dx.doi.org/10.1007/s13676-012-0010-0.
[4] L. Gouveia, P. Pesneau, M. Ruthmair, D. Santos, Combining and projecting flow models for the (precedence constrained) asymmetric traveling salesman problem, Networks 71 (2018) 451–465, http://dx.doi.org/10.1002/net.21765.
[5] J.A. Chisman, The clustered traveling salesman problem, Comput. Oper. Res. 2 (2) (1975) 115–119, http://dx.doi.org/10.1016/0305-0548(75)90015-5.
[6] T. Jenkyns, The greedy travelling salesman's problem, Networks 9 (1979) 363–373, http://dx.doi.org/10.1002/net.3230090406.
[7] Y. Dumas, J. Desrosiers, E. Gelinas, M.M. Solomon, An optimal algorithm for the traveling salesman problem with time windows, Oper. Res. 43 (2) (1995) 367–371, http://dx.doi.org/10.1287/opre.43.2.367.
[8] L. Bianco, A. Mingozzi, S. Ricciardelli, M. Spadoni, Exact and heuristic procedures for the traveling salesman problem with precedence constraints, based on dynamic programming, INFOR Inf. Syst. Oper. Res. 32 (1) (1994) 19–32, http://dx.doi.org/10.1080/03155986.1994.11732235.
[9] L. Gouveia, M. Ruthmair, Load-dependent and precedence-based models for pickup and delivery problems, Comput. Oper. Res. 63 (2015) 56–71, http://dx.doi.org/10.1016/j.cor.2015.04.008, URL https://www.sciencedirect.com/science/article/pii/S030505481500088X.
[10] D. Khachai, R. Sadykov, O. Battaia, M. Khachay, Precedence constrained generalized traveling salesman problem: Polyhedral study, formulations, and branch-and-cut algorithm, European J. Oper. Res. 309 (2) (2023) 488–505, http://dx.doi.org/10.1016/j.ejor.2023.01.039, URL https://www.sciencedirect.com/science/article/pii/S0377221723000735.
[11] S.L. Smith, F. Imeson, Glns: An effective large neighborhood search heuristic for the generalized traveling salesman problem, Comput. Oper. Res. 87 (2017) 1–19, http://dx.doi.org/10.1016/j.cor.2017.05.010, URL https://www.sciencedirect.com/science/article/pii/S0305054817301223.
[12] K. Helsgaun, An Extension of the Lin-Kernighan-Helsgaun Tsp Solver for Constrained Traveling Salesman and Vehicle Routing Problems: Technical Report, Roskilde University, 2017.
[13] H.H. Chieng, N. Wahid, A performance comparison of genetic algorithm's mutation operators in n-cities open loop travelling salesman problem, in: T. Herawan, R. Ghazali, M.M. Deris (Eds.), Recent Advances on Soft Computing and Data Mining, Springer International Publishing, 2014, pp. 89–97.
[14] F. Arnold, M. Gendreau, K. Sörensen, Efficiently solving very large-scale routing problems, Comput. Oper. Res. 107 (2019) 32–42, http://dx.doi.org/10.1016/j.cor.2019.03.006.
[15] R. Bellman, Dynamic programming treatment of the travelling salesman problem, J. ACM 9 (1) (1962) 61–63, http://dx.doi.org/10.1145/321105.321111.
[16] M. Held, R.M. Karp, A dynamic programming approach to sequencing problems, J. Soc. Ind. Appl. Math. 10 (1) (1962) 196–210, http://dx.doi.org/10.1137/0110015.
[17] M. de Berg, H.L. Bodlaender, S. Kisfaludi-Bak, S. Kolay, An ETH-tight exact algorithm for euclidean TSP, SIAM J. Comput. 52 (3) (2023) 740–760, http://dx.doi.org/10.1137/22M1469122.
[18] W. Cook, Concorde TSP solver, 2020, https://www.math.uwaterloo.ca/tsp/concorde.html. (Last Accessed 12 November 2023).
[19] H.H. Hoos, T. Stützle, On the empirical scaling of run-time for finding optimal solutions to the travelling salesman problem, European J. Oper. Res. 238 (1) (2014) 87–94, http://dx.doi.org/10.1016/j.ejor.2014.03.042.
[20] S. Hougardy, X. Zhong, Hard to solve instances of the euclidean traveling salesman problem, Math. Program. Comput. 13 (2021) 51–74, http://dx.doi.org/10.1007/s12532-020-00184-5.
[21] A. Ambainis, K. Balodis, J. Iraids, M. Kokainis, K. Prusis, J. Vihrovs, Quantum speedups for exponential-time dynamic programming algorithms, in: Proceedings of the 2019 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA, 2019, pp. 1783–1793, http://dx.doi.org/10.1137/1.9781611975482.107, arXiv:https://epubs.siam.org/doi/pdf/10.1137/1.9781611975482.107.
[22] J. Monnot, V. Paschos, S. Toulouse, Approximation algorithms for the traveling salesman problem, Math. Models Oper. Res. 145 (3) (2002) 537–548.
[23] N. Christofides, Worst-case analysis of a new heuristic for the travelling salesman problem, Oper. Res. Forum 3 (1) (2022) 20, http://dx.doi.org/10.1007/s43069-021-00101-z.
[24] A.R. Karlin, N. Klein, S.O. Gharan, A (slightly) improved approximation algorithm for metric TSP, in: Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 32–45, http://dx.doi.org/10.1145/3406325.3451009.
[25] S. Arora, Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems, J. ACM 45 (5) (1998) 753–782, http://dx.doi.org/10.1145/290179.290180.
[26] J.S.B. Mitchell, Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric tsp, k-mst, and related problems, SIAM J. Comput. 28 (4) (1999) 1298–1309, http://dx.doi.org/10.1137/S0097539796309764.
[27] S.B. Rao, W.D. Smith, Approximating geometrical graphs via spanners and banyans, in: Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing, STOC '98, Association for Computing Machinery, New York, NY, USA, 1998, pp. 540–550, http://dx.doi.org/10.1145/276698.276868.
[28] S. Kisfaludi-Bak, J. Nederlof, K. Wegrzycki, A gap-ETH-tight approximation scheme for euclidean TSP, in: 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science, FOCS, IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 351–362, http://dx.doi.org/10.1109/FOCS52979.2021.00043.
[29] Y. Bartal, L.-A. Gottlieb, A linear time approximation scheme for euclidean TSP, in: 2013 IEEE 54th Annual Symposium on Foundations of Computer Science, 2013, pp. 698–706, http://dx.doi.org/10.1109/FOCS.2013.80.

[30] Y. Bartal, L.-A. Gottlieb, R. Krauthgamer, The traveling salesman problem: Low-dimensionality implies a polynomial time approximation scheme, SIAM J. Comput. 45 (4) (2016) 1563–1581, http://dx.doi.org/10.1137/130913328.
[31] K. Helsgaun, An effective implementation of the Lin-Kernighan traveling salesman heuristic, European J. Oper. Res. 126 (1) (2000) 106–130, http://dx.doi.org/10.1016/S0377-2217(99)00284-2.
[32] K. Helsgaun, Using POPMUSIC for Candidate Set Generation in the Lin-Kernighan-Helsgaun TSP Solver, Tech. Rep. Computer Science, Roskilde University, 2018.
[33] S. Lin, B.W. Kernighan, An effective heuristic algorithm for the traveling-salesman problem, Oper. Res. 21 (1973) 498–516, http://dx.doi.org/10.1287/opre.21.2.498.
[34] U.A. Brodowsky, S. Hougardy, X. Zhong, The approximation ratio of the k-opt heuristic for the euclidean traveling salesman problem, SIAM J. Comput. 52 (4) (2023) 841–864, http://dx.doi.org/10.1137/21M146199X.
[35] D.J. Rosenkrantz, R.E. Stearns, P.M.L. II, An analysis of several heuristics for the traveling salesman problem, SIAM J. Comput. 6 (3) (1977) 563–581.
[36] K. Panwar, K. Deep, Transformation operators based grey wolf optimizer for travelling salesman problem, J. Comput. Sci. 55 (2021) 101454, http://dx.doi.org/10.1016/j.jocs.2021.101454.
[37] P. McMenemy, N. Veerapen, J. Adair, G. Ochoa, Rigorous performance analysis of state-of-the-art tsp heuristic solvers, in: A. Liefooghe, L. Paquete (Eds.), Evolutionary Computation in Combinatorial Optimization, Springer International Publishing, Cham, 2019, pp. 99–114.
[38] S.N. Bespamyatnikh, An optimal algorithm for closest-pair maintenance, Discrete Comput. Geom. 19 (1998) 175–195, http://dx.doi.org/10.1007/PL00009340.
[39] S. de Berg, F. Staals, Dynamic data structures for k-nearest neighbor queries, Comput. Geom. 111 (2023) 101976, http://dx.doi.org/10.1016/j.comgeo.2022.101976.
[40] N. Beckmann, H.-P. Kriegel, R. Schneider, B. Seeger, The R*-tree: An efficient and robust access method for points and rectangles, in: Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, SIGMOD '90, Association for Computing Machinery, New York, NY, USA, 1990, pp. 322–331, http://dx.doi.org/10.1145/93597.98741.
[41] B. organization, Boost C++ libraries, 2023, http://www.boost.org. (Last Accessed 20 May 2023).
[42] M. Held, R.M. Karp, The traveling-salesman problem and minimum spanning trees, Oper. Res. 18 (6) (1970) 1138–1162.
[43] D.S. Johnson, L.A. McGeoch, E.E. Rothberg, Asymptotic experimental analysis for the held–karp traveling salesman bound, in: Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '96, Society for Industrial and Applied Mathematics, USA, 1996, pp. 341–350.
[44] G. Reinelt, TSPLIB–A traveling salesman problem library, ORSA J. Comput. 3 (4) (1991) 376–384, http://dx.doi.org/10.1287/ijoc.3.4.376.
[45] W. Cook, Traveling salesman problem, 2023, https://www.math.uwaterloo.ca/tsp/index.html. (Last Accessed 20 May 2023).
[46] 8Th DIMACS implementation challenge: The traveling salesman problem, 2023, http://dimacs.rutgers.edu/archive/Challenges/TSP. (Last Accessed 27 August 2023).
[47] P. Fränti, R. Mariescu-Istodor, Santa claus TSP challenge, 2020, https://cs.uef.fi/sipu/santa/. (Last Accessed 27 August 2023).
[48] R. Mariescu-Istodor, P. Fränti, Solving the large-scale TSP problem in 1h: Santa claus challenge 2020, Front. Robotics AI 8 (2021) http://dx.doi.org/10.3389/frobt.2021.689908.
[49] L. Sengupta, R. Mariescu-Istodor, P. Fränti, Which local search operator works best for the open-loop TSP? Appl. Sci. 9 (19) (2019) 1–24, http://dx.doi.org/10.3390/app9193985.
[50] P. Fränti, T. Nenonen, M. Yuan, Converting MST to TSP path by branch elimination, Appl. Sci. 11 (1) (2021) http://dx.doi.org/10.3390/app11010177.
[51] P. Fränti, R. Mariescu-Istodor, MOPSI dots 2017, 2017, http://cs.uef.fi/o-mopsi/datasets/dots/. (Last Accessed 27 August 2023).
[52] A. Jazayeri, H. Sayama, A polynomial-time deterministic approach to the travelling salesperson problem, Int. J. Parallel Emergent Distrib. Syst. 35 (4) (2020) 454–460.
[53] S. Wang, W. Rao, Y. Hong, A distance matrix based algorithm for solving the traveling salesman problem, Oper. Res. 20 (3) (2020) 1505–1542, http://dx.doi.org/10.1007/s12351-018-0386-1.
[54] V. Pacheco-Valencia, J.A. Hernández, J.M. Sigarreta, N. Vakhania, Simple constructive, insertion, and improvement heuristics based on the girding polygon for the euclidean traveling salesman problem, Algorithms 13 (1) (2020) 1–30, http://dx.doi.org/10.3390/a13010005.
[55] E.D. Taillard, A linearithmic heuristic for the travelling salesman problem, European J. Oper. Res. 297 (2) (2022) 442–450, http://dx.doi.org/10.1016/j.ejor.2021.05.034.
[56] T. Strutz, Travelling santa problem: Optimization of a million-households tour within one hour, Front. Robotics AI 8 (2021) 1–14, http://dx.doi.org/10.3389/frobt.2021.652417.
[57] T. Strutz, Redesigning the wheel for systematic travelling salesmen, Algorithms 16 (2) (2023) 1–33, http://dx.doi.org/10.3390/a16020091.
[58] F. Uddin, N. Riaz, A. Manan, I. Mahmood, O.-Y. Song, A.J. Malik, A.A. Abbasi, An improvement to the 2-opt heuristic algorithm for approximation of optimal TSP tour, Appl. Sci. 13 (12) (2023) 1–24, http://dx.doi.org/10.3390/app13127339.
[59] J. Bartholdi, L. Platzman, An $O(n \log n)$ planar travelling salesman heuristic based on spacefilling curves, Oper. Res. Lett. 1 (4) (1982) 121–125, http://dx.doi.org/10.1016/0167-6377(82)90012-8.
[60] D.S. Johnson, L.A. McGeoch, The traveling salesman problem: A case study in local optimization, in: E.H.L. Aarts, J.K. Lenstra (Eds.), Local Search in Combinatorial Optimization, John Wiley and Sons, Chichester, United Kingdom, 1997, pp. 215–310.

Arno Formella has received his academic formation at the University of the Saarland, Germany, achieving his Master's Degree in 1989 and his Ph.D. in 1993. Since 1999, he is a professor of the Computer Science Department at the University of Vigo, Spain, teaching at the School of Computer Science Engineering in undergraduate and postgraduate courses. His research interests lie in the field of applied computer science in areas such as physics, natural sciences, telecommunication, or space engineering, especially in the application of concurrent and parallel computing, computational geometry, particle simulation, image processing, pattern recognition, optimization, and software engineering.